

RÉSIDENCE AUREA

DOSSIER TECHNIQUE & ARCHITECTURE SYSTÈME

Plateforme Web Immobilière & Conseiller Commercial IA (RAG Nemotron-3 2048D / Llama 3.1 70B)

Document récapitulatif à destination de l'équipe technique et commerciale

1. VUE D'ENSEMBLE DE L'ARCHITECTURE

La plateforme Résidence Aurea est une application web moderne 'phygitale' combinant la présentation de biens immobiliers de luxe et un conseiller commercial virtuel IA disponible 24/7. Elle permet d'attirer, de qualifier et de convertir automatiquement les visiteurs en rendez-vous de visite privée.

Flux de Traitement IA & RAG (Architecture Réseau) :

- Visiteur Client !' Navigation & Chat Stream SSE (/api/chat)
- Moteur d'Embedding !' NVIDIA Nemotron-3-Embed-1B (Calcul du vecteur 2048D)
- Base Vectorielle !' PostgreSQL + pgvector (Recherche par Cosinus, Top 3 Chunks)
- Génération LLM !' NVIDIA NIM Llama 3.1 70B Instruct (Outillage Function Calling & Stream)
- CRM & Qualification !' Qualification asynchrone des Leads (HOT / WARM / COLD) en TND

2. STACK TECHNOLOGIQUE (TECHNOLOGIES UTILISÉES)

A. Frontend (Interface Utilisateur)

- Framework** : Next.js 14+ (App Router, Server Components & Client Components).
- Bibliothèque UI** : React 18 (Architecture réactive moderne).

Utilisation des Hooks React :

- **useState & useReducer** : Gestion de l'état réactif local et global (filtres du catalogue, formulaires, flux de chat, modales Admin).
- **useEffect** : Synchronisation des effets de bord (chargement de la session localStorage, pré-qualification, auto-scroll).
- **useRef** : Manipulation directe du DOM (défilement automatique du fil de discussion de l'agent IA, conteneurs de carte Leaflet).
- **useSearchParams & usePathname** : Synchronisation en temps réel des filtres de recherche et des intentions de réservation avec l'URL.
- **useContext** : Diffusion globale de la langue (i18n) et de l'orientation d'affichage (LTR/RTL).
- Langage** : TypeScript (Type strict de bout en bout).
- Styles & Design System** : Tailwind CSS + effets de Glassmorphism UI (Slate sombre #0f172a & Accents Dorés #d97706).
- Icônes & Visuels** : Lucide React & Génération d'artefacts visuels immobiliers haute résolution.
- Cartographie** : Leaflet / OpenStreetMap (Intégration dynamique des coordonnées GPS des projets).
- Internationalisation (i18n)** : Prise en charge multilingue (Français, Anglais, Arabe avec basculement automatique du layout en RTL dir='rtl').

B. Backend & API Infrastructure

- Runtime** : Node.js runtime unifié avec Next.js App Router.
- Streaming d'IA** : Server-Sent Events (SSE) via ReadableStream pour l'affichage fluide et progressif des réponses du conseiller virtuel.
- Authentification Sécurisée** : NextAuth (Provider Credentials pour l'Espace Administrateur).
- CMS Visual Customizer** : Éditeur dynamique avec panneau de prévisualisation en temps réel via iframe et événements postMessage.

C. Moteur d'Intelligence Artificielle & RAG

- **Modèle d'Embedding** : nvidia/nemotron-3-embed-1b via l'infrastructure NVIDIA NIM (Modèle 1B paramètres produisant des vecteurs de 2048 dimensions).
- **Modèle de Langage (LLM)** : meta/llama-3.1-70b-instruct via l'API OpenAI-compatible NVIDIA NIM.
- **Moteur RAG Vectoriel** : Recherche par similarité cosinus directement en SQL natif via l'extension PostgreSQL pgvector (1 - (embedding <=> query::vector)).
- **Optimisation de Latence** : Détection automatique des salutations (réponse instantanée < 150 ms) et sélection du Top 3 des fragments pertinents.
- **Suite d'Outils Agent (Function Calling)** : search_apartments, get_apartment_details, get_documents, get_available_slots, create_appointment, escalate_to_human.

D. Base de Données & Persistance

- **SGBD** : PostgreSQL (Version 15+).
- **Extension Vectorielle** : pgvector (prise en charge des index vectoriels 2048 dimensions).
- **ORM** : Prisma 7 (@prisma/client avec adaptateur de pool natif @prisma/adapter-pg).
- **Devise Principale** : Dinar Tunisien (TND / DT) appliqué sur l'ensemble du catalogue, du simulateur de prêt et de l'agent IA.

E. Notifications & E-Mails

- **Générateur iCalendar (.ics)** : Formatage UTC automatique pour l'envoi de fichiers d'invitation de rendez-vous dans les agendas clients (Google Calendar, Outlook, Apple Calendar).
- **Fournisseur E-mail** : Provider générique (Resend / SMTP).

3. SCHÉMA DES DONNÉES (MODÈLES PRISMA PRINCIPATION)

- **Project** : Programme immobilier (Nom, localisation GPS, photos, statut).
- **Apartment** : Appartements et attiques (Référence, prix en TND, surface m², pièces, orientation, étage, statut).
- **Document & DocumentChunk** : Brochures PDF, plans 3D et leurs vecteurs d'indexation 2048D.
- **Lead** : Prospects qualifiés (Nom, email, téléphone, score HOT/WARM/COLD, budget min/max, urgence).
- **Conversation & Message** : Historique des échanges entre le client et l'agent IA.
- **Appointment** : Rendez-vous de visites planifiés (Créneau ISO, type de visite, statut d'approbation).
- **CmsSection & SiteSettings** : Réglages graphiques, polices, couleurs et contenus dynamiques de la landing page.

4. PROCÉDURE D'INSTALLATION & DÉPLOIEMENT

Prérequis système : Node.js 18+, Docker & Docker Compose, Clé API NVIDIA NIM.

Étape 1 : Lancement de la Base de Données PostgreSQL (Docker)

```
docker compose up -d
```

Étape 2 : Synchronisation du Schéma & Seed Initial

```
npx prisma db push
npx tsx prisma/seed.ts
```

Étape 3 : Réindexation Vectorielle RAG (Nemotron-3 2048D)

```
npx tsx scripts/reindex-all.ts
```

Étape 4 : Lancement du Serveur de Développement

```
npm run dev  
# Accès à l'application sur http://localhost:3001
```

5. DÉPLOIEMENT EN PRODUCTION (CONTAINER DOCKER)

Un fichier Dockerfile multi-étapes (multi-stage) optimisé est fourni à la racine du projet pour la mise en production :

- **Étape 1 - deps** : Installation des dépendances de production.
 - **Étape 2 - builder** : Génération du client Prisma et compilation Next.js optimisée (output: 'standalone').
 - **Étape 3 - runner** : Image Alpine Linux ultra-légère exécutant le serveur de production autonome server.js.
-

